# Decisiones Técnicas y Justificación de Tecnologías Seleccionadas:

# Plataforma AdoPaws

Albany Gonzalez.

José Pablo Monestel.

Avril Madrigal.

Rebeca Arce.

Universidad Invenio

Ingeniería de Software 1

Darin Mauricio Gamboa

25 de marzo de 2026

**Resumen**

El presente documento expone las decisiones técnicas adoptadas para el desarrollo de AdoPaws, una plataforma web orientada a la adopción de mascotas, la comercialización de artículos relacionados y la consulta con profesionales veterinarios. Se justifica la selección de cada tecnología en función de los requisitos arquitectónicos del sistema, los principios de Clean Architecture y las necesidades de calidad, escalabilidad y mantenibilidad. El stack tecnológico comprende React 18 con Vite en el frontend, ASP.NET Core Web API con .NET en el backend, SQL Server como gestor de base de datos relacional, Entity Framework Core como ORM, la API de Anthropic Claude para inteligencia artificial, xUnit y Vitest para pruebas automatizadas, GitHub Actions para integración continua y SonarQube para inspección continua de calidad de código.

*Palabras clave:* stack tecnológico, Clean Architecture, ASP.NET Core, React, Entity Framework Core, CI/CD, SonarQube, decisiones de diseño.

# Introducción

La selección de tecnologías en un proyecto de software constituye una decisión de alto impacto que determina la capacidad del sistema para cumplir sus requisitos funcionales y no funcionales a lo largo de su ciclo de vida. En el contexto de AdoPaws, esta decisión debía satisfacer simultáneamente la necesidad de construir una interfaz de usuario moderna y reactiva, una API RESTful robusta y organizada, una base de datos relacional íntegra y un ciclo DevOps completo con pruebas automatizadas e inspección de calidad.

El presente documento detalla cada tecnología seleccionada, describe su rol en la arquitectura del sistema, presenta evidencia técnica de su implementación concreta en el proyecto y justifica su elección frente a alternativas disponibles. El objetivo es que este documento sirva como registro formal de las decisiones de ingeniería adoptadas y como referencia para el mantenimiento y evolución futura del sistema.

# Arquitectura del Sistema

AdoPaws implementa Clean Architecture organizada en cuatro capas con dependencias unidireccionales hacia el dominio central. Esta arquitectura garantiza que los cambios en la infraestructura (base de datos, servicios externos) no afecten la lógica de negocio, y que la lógica de negocio no esté acoplada a detalles de presentación.

## Capa de Dominio (Adopaws.Domain)

Contiene las entidades puras del sistema sin dependencias externas: User, Pet, PetPhoto, AdoptionRequest, MarketplaceItem, Consultation y ConsultationResponse. Las entidades definen los atributos del dominio y las navigation properties que expresan las relaciones entre entidades. Ninguna clase de esta capa hace referencia a Entity Framework Core, a la base de datos ni a ningún servicio externo, garantizando la independencia tecnológica del modelo de dominio.

Ejemplo de entidad de dominio: la entidad Pet define los campos IdPet, IdUser, Name, PetType, Breed, Age, Gender, Size, Vaccinated, Sterilized, Description, Region, PublicationStatus y PublishedDate, junto con navigation properties hacia User, PetPhotos y AdoptionRequests. Este diseño permite que la capa de aplicación opere sobre objetos del dominio sin conocer los detalles de persistencia.

**Capa de Aplicación (Adopaws.Application)**

Contiene los contratos del sistema (interfaces de repositorios y servicios), los Data Transfer Objects (DTOs) para la comunicación entre capas, los mappers que transforman entidades en DTOs y los servicios que implementan los casos de uso. La capa de aplicación depende únicamente de la capa de dominio, cumpliendo el principio de inversión de dependencias.

Los servicios implementan la lógica de negocio: UserService gestiona el ciclo de vida de los usuarios, PetService administra las publicaciones de mascotas, AdoptionRequestService orquesta el flujo de solicitudes y CompatibilityService calcula el score de afinidad usuario-mascota integrando el motor de reglas y la API de inteligencia artificial.

**Capa de Infraestructura (Adopaws.Infrastructure)**

Implementa los contratos de repositorios definidos en la capa de aplicación mediante Entity Framework Core. Contiene el AdopawsDbContext, las configuraciones Fluent API por entidad (IEntityTypeConfiguration<T>), las migraciones de base de datos y el módulo de inyección de dependencias (InfrastructureServiceExtensions) que registra todos los repositorios y servicios en el contenedor de IoC de ASP.NET Core con ciclo de vida Scoped.

**Capa de Presentación (Adopaws.Api)**

Expone los endpoints REST mediante controladores delgados que únicamente validan la entrada y delegan la ejecución al servicio correspondiente. Incluye el middleware global de manejo de excepciones (ExceptionHandlingMiddleware) que intercepta todas las excepciones no controladas y retorna respuestas JSON estandarizadas con statusCode, message y timestamp. La documentación automática de la API se genera mediante Swagger/OpenAPI configurado en SwaggerExtensions.

Tabla 1

*Capas de la Arquitectura y sus Dependencias*

| Capa | Proyecto | Depende de | Responsabilidad Principal |
|---|---|---|---|
| **Dominio** | Adopaws.Domain | Ninguna | Entidades puras y contratos del dominio |
| **Aplicación** | Adopaws.Application | Domain | Casos de uso, DTOs, interfaces, mappers |
| **Infraestructura** | Adopaws.Infrastructure | Application, Domain | EF Core, repositorios, migraciones |
| **Presentación** | Adopaws.Api | Application, Infrastructure | Controladores REST, middleware, Swagger |

*Nota.* La dirección de las dependencias apunta siempre hacia la capa de Dominio, garantizando la independencia tecnológica del núcleo del sistema.

## Tecnologías Seleccionadas y Justificación

### Frontend: React 18 con Vite

React es una biblioteca de JavaScript para la construcción de interfaces de usuario basadas en componentes reutilizables y un modelo de actualización declarativo del DOM (Meta Open Source, 2024). Se seleccionó React 18 como framework de frontend por su madurez, ecosistema consolidado y compatibilidad con la arquitectura SPA requerida por el proyecto.

Vite fue adoptado como herramienta de construcción en lugar de Create React App por su velocidad de inicio en desarrollo (Hot Module Replacement nativo), tiempos de build significativamente menores y soporte nativo para módulos ES. En la práctica, el proyecto utiliza npm run dev para el servidor de desarrollo en http://localhost:5173 y npm run build para la generación del bundle de producción en el directorio /dist.

La gestión del estado de autenticación se implementó mediante React Context API (AuthContext.jsx), evitando la complejidad de Redux para el alcance del proyecto. El enrutamiento del lado del cliente se gestiona con React Router v6, que permite la definición de rutas protegidas mediante el componente de orden superior ProtectedRoute, que valida la existencia del token JWT y el rol del usuario antes de renderizar la ruta solicitada.

**Backend: ASP.NET Core Web API con .NET**

ASP.NET Core es el framework de desarrollo web de código abierto de Microsoft para .NET, diseñado para construir APIs RESTful de alto rendimiento y multiplataforma (Microsoft, 2024). Se seleccionó como framework de backend por su integración nativa con Entity Framework Core, su sistema de inyección de dependencias incorporado, su middleware pipeline configurable y su soporte para el modelo de programación asíncrona (async/await) que optimiza el uso de recursos en operaciones de I/O.

El backend se estructura como un monolito modular con arquitectura de capas. Program.cs registra los servicios de infraestructura mediante builder.Services.AddInfrastructure(builder.Configuration), configura el middleware de CORS con política AllowAll para el entorno de desarrollo, registra el middleware global de manejo de excepciones y configura Swagger UI. Los controladores heredan de ControllerBase y utilizan el atributo [ApiController] para habilitar validaciones automáticas de model binding.

**Base de Datos: SQL Server**

SQL Server es el sistema de gestión de bases de datos relacionales de Microsoft, seleccionado por su integración nativa con Entity Framework Core y ASP.NET Core, su soporte para transacciones ACID, sus herramientas de administración consolidadas y su disponibilidad en edición Developer gratuita para entornos de desarrollo (Microsoft, 2024).

La base de datos AdopawsDB es creada y gestionada íntegramente mediante migraciones de EF Core. La cadena de conexión se configura en appsettings.json bajo la clave DefaultConnection con el formato: Server=.;Database=AdopawsDB;Trusted_Connection=True;TrustServerCertificate=True. Para instancias de SQL Server Express, el servidor se especifica como .\SQLEXPRESS.

**ORM: Entity Framework Core**

Entity Framework Core es el ORM oficial de Microsoft para .NET que permite trabajar con bases de datos mediante objetos C# (Microsoft, 2024). Se seleccionó como capa de acceso a datos por su integración profunda con ASP.NET Core, su soporte para el patrón Repository mediante DbContext, su sistema de migraciones que versionan el esquema de base de datos de forma reproducible y su capacidad para generar consultas SQL optimizadas mediante LINQ.

Las entidades se configuran mediante el patrón Fluent API en clases IEntityTypeConfiguration<T> separadas por entidad. Esta separación mantiene las clases de entidad limpias de atributos de configuración y centraliza las reglas de mapeo. Las configuraciones relevantes incluyen: índice único sobre Users.Email para garantizar la unicidad del correo electrónico, índice único compuesto sobre AdoptionRequests(IdPet, IdUser) para prevenir solicitudes duplicadas, relación hasOne/withMany entre Pet y User con DeleteBehavior.Cascade y relación de Consultation con dos foreign keys hacia User (SenderIdUser y ReceiverIdUser) configuradas con DeleteBehavior.Restrict para evitar eliminaciones en cascada no deseadas.

**Inteligencia Artificial: API de Anthropic Claude**

La API de Anthropic Claude es un servicio de inteligencia artificial que expone modelos de lenguaje de gran escala mediante una API REST. Se integró en el módulo de compatibilidad de AdoPaws para generar explicaciones personalizadas de la afinidad entre un usuario y una mascota, enriqueciendo el motor de puntuación basado en reglas con interpretaciones en lenguaje natural.

La integración utiliza IHttpClientFactory con un cliente nombrado AnthropicClient, registrado en InfrastructureServiceExtensions. La API Key se almacena en la sección Anthropic:ApiKey de appsettings.json. El sistema implementa degradación elegante: ante cualquier fallo del servicio externo, el CompatibilityService retorna automáticamente la explicación generada por el motor de reglas sin interrumpir la experiencia del usuario.

**Testing: xUnit y Vitest**

xUnit es el framework de pruebas unitarias estándar para .NET, seleccionado por su integración nativa con el ecosistema dotnet test, su compatibilidad con el reporte de cobertura en formato Cobertura XML requerido por SonarQube y su soporte para pruebas asíncronas mediante async/await. Las pruebas del backend se organizan por servicio, utilizando Moq para la creación de repositorios simulados que aislan la lógica de negocio de la base de datos.

Vitest es el framework de pruebas para el frontend, seleccionado por su compatibilidad nativa con el entorno de construcción Vite, sus tiempos de ejecución significativamente menores

respecto a Jest en proyectos Vite y su API compatible con Jest que facilita la migración. Se combina con React Testing Library para pruebas centradas en el comportamiento observable del usuario más que en los detalles de implementación de los componentes.

**CI/CD: GitHub Actions**

GitHub Actions es la plataforma de automatización integrada en GitHub, seleccionada como herramienta de CI/CD por su integración nativa con el repositorio del proyecto, su modelo basado en archivos YAML versionados junto al código fuente, su amplio catálogo de acciones reutilizables y su disponibilidad gratuita para repositorios públicos (GitHub, 2024). El pipeline del proyecto implementa cinco stages secuenciales: Build, Test, SonarQube Analysis, Security Check y Deploy Staging, garantizando que únicamente código que supera todos los controles de calidad sea integrado a la rama principal.

**Calidad de Código: SonarQube / SonarCloud**

SonarQube es la plataforma líder de inspección continua de calidad de código, que analiza métricas de cobertura, duplicación, mantenibilidad, confiabilidad y seguridad (SonarSource, 2024). Se integra como tercer stage del pipeline de CI/CD, recibiendo los reportes de cobertura generados por xUnit para calcular métricas consolidadas. Los Quality Gates configurados establecen umbrales mínimos que el proyecto debe superar para que el pipeline apruebe: cobertura mayor o igual al 70%, duplicación menor al 3%, ratings de mantenibilidad y confiabilidad A o B, rating de seguridad A y cero bugs críticos o blockers.

Tabla 2

*Resumen de Tecnologías Seleccionadas por Capa*

| Capa / Área | Tecnología | Versión | Justificación Principal |
|---|---|---|---|
| **Frontend** | React + Vite | 18 / 5.x | SPA reactiva, componentes reutilizables, HMR rápido |
| **Backend** | ASP.NET Core Web API | .NET 10 | API REST robusta, DI nativa, async/await, middleware pipeline |
| **Base de datos** | SQL Server | 2019+ | ACID, integración EF Core, edición Developer gratuita |
| **ORM** | Entity Framework Core | 10.x | Migraciones versionadas, Fluent API, LINQ, patrón Repository |
| **IA** | Anthropic Claude API | claude-sonnet-4 | Explicaciones en lenguaje natural, degradación elegante |
| **Testing Backend** | xUnit + Moq | 2.x | Estándar .NET, soporte async, cobertura Cobertura XML |
| **Testing Frontend** | Vitest + RTL | 1.x | Nativo Vite, API Jest-compatible, pruebas centradas en usuario |
| **CI/CD** | GitHub Actions | N/A | Integración nativa GitHub, YAML versionado, gratuito OSS |
| **Calidad** | SonarQube/SonarCloud | Community | Quality Gates, cobertura, mantenibilidad, seguridad |

| | | | |
|---|---|---|---|
| **Control versiones** | Git + GitHub | N/A | Branching strategy, Pull Requests, historial de cambios |

*Nota.* RTL = React Testing Library. OSS = Open Source Software.

## Conclusión

Las decisiones técnicas adoptadas en AdoPaws constituyen un conjunto coherente y justificado que responde a los requisitos funcionales, no funcionales y de calidad del proyecto. La adopción de Clean Architecture garantiza la independencia tecnológica del dominio y facilita la evolución del sistema. La combinación React + ASP.NET Core + SQL Server provee una base madura y bien documentada para el desarrollo de aplicaciones web empresariales.

La integración de la API de Anthropic Claude aporta diferenciación funcional mediante el módulo de compatibilidad inteligente. La incorporación del ciclo DevOps completo (xUnit, Vitest, GitHub Actions, SonarQube) eleva el nivel de madurez del proyecto desde un prototipo funcional hacia un producto de software mantenible y verificable. En conjunto, estas decisiones permiten construir un sistema organizado, eficiente y preparado para escalar hacia versiones futuras.

**Referencias**


- GitHub. (2024). *GitHub Actions documentation*. https://docs.github.com/en/actions
- Martin, R. C. (2017). *Clean architecture: A craftsman's guide to software structure and design*. Prentice Hall.
- Meta Open Source. (2024). *React documentation*. https://react.dev/
- Microsoft. (2024). *ASP.NET Core documentation*. https://learn.microsoft.com/en-us/aspnet/core/
- Microsoft. (2024). *Entity Framework Core documentation*. https://learn.microsoft.com/en-us/ef/core/
- Microsoft. (2024). *SQL Server documentation*. https://learn.microsoft.com/en-us/sql/sql-server/
- SonarSource. (2024). *SonarQube documentation*. https://docs.sonarsource.com/sonarqube/